PRINCIPLES OF MACHINE LEARNING LAB

Common Tasks 1–7 and Topic-Specific Tasks 8–11

Practical preparation guide • Python / scikit-learn • With print outputs

Included topics (9): SVM, MLP / Neural Network, Decision Tree, Random Forest, KNN, K-Means, Linear Regression, Bagging, and PCA.

Use the common preprocessing templates where appropriate, then choose the matching algorithm section for Tasks 8–11. Output values depend on your dataset and will not exactly match the examples.

Prepared as a reusable lab reference. Update dataset filenames and target-column names to match the files provided in your practical exam.

How to use this guide

The first five topics are supervised classification. K-Means is unsupervised clustering, Linear Regression predicts a continuous numerical value, Bagging is an ensemble classifier in the example below, and PCA reduces dimensionality rather than predicting a target.

Important: Tasks 1–7 are not literally identical for every topic. Use the shared classification template for SVM, MLP, Decision Tree, Random Forest, and KNN. K-Means and PCA do not require a target column; Linear Regression requires a numerical target. Bagging uses classification labels in this guide.

Topic map

No.	Topic	Task 8	Task 9	Task 10	Task 11
1	SVM	Train model	Evaluate	Compare kernels	5-fold CV
2	MLP / Neural Network	Train model	Evaluate	Compare hidden layers	5-fold CV
3	Decision Tree	Train model	Evaluate	Compare max depth	5-fold CV
4	Random Forest	Train model	Evaluate	Compare tree counts	5-fold CV
5	KNN	Train model	Evaluate	Compare K values	5-fold CV
6	K-Means	Cluster data	Evaluate clusters	Compare K values	Silhouette score
7	Linear Regression	Train model	Regression metrics	Compare models	5-fold CV
8	Bagging	Train model	Evaluate	Compare estimator counts	5-fold CV
9	PCA	Reduce dimensions	Visualize	Explained variance	Cumulative variance

Common Tasks 1–7: Classification template

Use this section for SVM, MLP, Decision Tree, Random Forest, and KNN. Change DATA_FILE to the dataset file. The target detector recognizes common names, but if your dataset uses a different label column, set TARGET manually.

Task 1 — Load dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

DATA_FILE = "diabetes.csv"   # Change this filename
df = pd.read_csv(DATA_FILE)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\nColumn names:")
print(df.columns.tolist())
```

Task 2 — Inspect data and target distribution

```
df.columns = df.columns.str.strip()

target_candidates = [
    "class", "label", "target", "outcome", "attrition"
]
target = next(
    (col for col in df.columns if col.lower() in target_candidates),
    None
)

if target is None:
    print("Available columns:", df.columns.tolist())
    raise ValueError("Set target to the correct target-column name.")

print("Target column:", target)
print("\nDataset information:")
df.info()
print("\nMissing values per column:")
print(df.isnull().sum())
print("\nClass distribution:")
print(df[target].value_counts())

df[target].value_counts().plot(kind="bar")
plt.title("Class Distribution")
plt.xlabel(target)
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

Task 3 — Remove irrelevant columns and missing targets

```
df = df.drop(
    columns=[
        "ID", "EmployeeNumber", "EmployeeCount",
        "Over18", "StandardHours"
    ],
    errors="ignore"
)
df = df.dropna(subset=[target]).copy()

print("Irrelevant columns removed (where present).")
print("Rows with missing target removed.")
print("\nDataset after cleaning:")
print(df.head())
print("\nDataset shape:", df.shape)
print("Remaining columns:", df.columns.tolist())
```

Task 4 — Separate features X and target y

```
X = df.drop(columns=[target]).copy()
y = df[target].astype(str).str.strip()

print("Features (X) created successfully.")
print("Target (y) created successfully.")
print("\nFirst 5 feature rows:")
print(X.head())
print("\nFirst 5 target values:")
print(y.head().tolist())
print("\nX shape:", X.shape)
print("y shape:", y.shape)
```

Task 5 — Encode categorical features

```
X = pd.get_dummies(X, drop_first=True, dtype=int)

print("Categorical features encoded successfully.")
print("\nFirst 5 encoded rows:")
print(X.head())
print("\nEncoded feature shape:", X.shape)
print("Remaining categorical columns:",
      X.select_dtypes(include=["object", "category"]).columns.tolist())
```

Task 6 — Split into training and testing sets

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Dataset split successfully.")
print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

Task 7 — Scale features (for SVM, MLP, and KNN)

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Feature scaling completed successfully!")
print("Scaled training shape:", X_train.shape)
print("Scaled testing shape :", X_test.shape)
print("\nFirst 5 scaled training samples:")
print(X_train[:5])
```

For Decision Tree, Random Forest, and Bagging, scaling is usually not necessary. For those algorithms, skip the scaling cell or keep an unscaled copy of the encoded X_train and X_test. Do not run the same scaling cell repeatedly.

Topic 1 — SVM (Support Vector Machine)

Example dataset: diabetes.csv. Tasks 8–11 assume Tasks 1–7 have prepared and scaled X_train, X_test, y_train, and y_test.

Task 8 — Train SVM and predict

```
from sklearn.svm import SVC

svm_model = SVC(kernel="linear", random_state=42)
svm_model.fit(X_train, y_train)
y_pred = svm_model.predict(X_test)

print("SVM model trained successfully!")
print("Kernel:", "linear")
print("Training samples:", X_train.shape[0])
print("Number of features:", X_train.shape[1])
print("\nFirst 10 actual values:", y_test[:10].tolist())
print("First 10 predicted values:", y_pred[:10].tolist())
```

Task 9 — Evaluate SVM

```
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    ConfusionMatrixDisplay, precision_score, recall_score, f1_score
)
import matplotlib.pyplot as plt

print("SVM Model Evaluation")
print("--------------------")
print(f"Accuracy  : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"Recall    : {recall_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"F1-Score  : {f1_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("SVM Confusion Matrix")
plt.tight_layout()
plt.show()
```

Task 10 — Compare SVM kernels

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

results = {}
print("SVM Kernel Comparison")
for kernel in ["linear", "poly", "rbf"]:
    model = SVC(kernel=kernel, random_state=42)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    results[kernel] = accuracy_score(y_test, pred)
    print(f"{kernel} kernel accuracy: {results[kernel]:.4f}")

best_kernel = max(results, key=results.get)
print("\nBest kernel:", best_kernel)
print("Best accuracy:", f"{results[best_kernel]:.4f}")
```

Task 11 — 5-fold cross-validation

```
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
import numpy as np

model = make_pipeline(StandardScaler(), SVC(kernel="linear"))
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
print("SVM Cross-Validation Results")
for i, score in enumerate(scores, 1):
    print(f"Fold {i} accuracy: {score:.4f}")
print("Mean accuracy:", f"{np.mean(scores):.4f}")
print("Standard deviation:", f"{np.std(scores):.4f}")
```

The CV pipeline scales inside each fold. If X_train was already scaled, this adds a second scaling step; ideally keep the unscaled split for pipeline-based cross-validation.

Topic 2 — MLP / Neural Network

MLP is a neural-network classifier. It usually benefits from scaled numerical features.

Task 8 — Train MLP

```
from sklearn.neural_network import MLPClassifier

mlp_model = MLPClassifier(
    hidden_layer_sizes=(100,), activation="relu",
    solver="adam", max_iter=500, early_stopping=True,
    random_state=42
)
mlp_model.fit(X_train, y_train)
y_pred = mlp_model.predict(X_test)

print("MLP model trained successfully!")
print("Hidden layers:", mlp_model.hidden_layer_sizes)
print("Activation:", mlp_model.activation)
print("Optimizer:", mlp_model.solver)
print("Iterations used:", mlp_model.n_iter_)
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("\nFirst 10 actual values:", y_test[:10].tolist())
print("First 10 predicted values:", y_pred[:10].tolist())
```

Task 9 — Evaluate MLP

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

print("MLP Model Evaluation")
print(f"Accuracy  : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"Recall    : {recall_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"F1-Score  : {f1_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("MLP Confusion Matrix")
plt.tight_layout()
plt.show()
```

Task 10 — Compare hidden-layer configurations

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

results = {}
for layers in [(50,), (100,), (50, 25)]:
    model = MLPClassifier(
        hidden_layer_sizes=layers, activation="relu",
        solver="adam", max_iter=500, early_stopping=True,
        random_state=42
    )
    model.fit(X_train, y_train)
    score = accuracy_score(y_test, model.predict(X_test))
    results[layers] = score
    print(f"Hidden layers {layers}: accuracy = {score:.4f}")

best = max(results, key=results.get)
print("\nBest configuration:", best)
print("Best accuracy:", f"{results[best]:.4f}")
```

Task 11 — 5-fold cross-validation

```
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
import numpy as np

model = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(100,), max_iter=500,
                  early_stopping=True, random_state=42)
)
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
print("MLP Cross-Validation Results")
for i, score in enumerate(scores, 1):
    print(f"Fold {i} accuracy: {score:.4f}")
print("Mean accuracy:", f"{np.mean(scores):.4f}")
print("Standard deviation:", f"{np.std(scores):.4f}")
```

Topic 3 — Decision Tree

Decision Trees do not normally require feature scaling. Use encoded numerical features from Task 5 and skip Task 7 scaling for this topic.

Task 8 — Train Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(
    criterion="gini", max_depth=5, random_state=42
)
dt_model.fit(X_train, y_train)
y_pred = dt_model.predict(X_test)

print("Decision Tree model trained successfully!")
print("Criterion:", dt_model.criterion)
print("Maximum depth:", dt_model.max_depth)
print("Actual tree depth:", dt_model.get_depth())
print("Number of leaves:", dt_model.get_n_leaves())
print("\nFirst 10 actual values:", y_test[:10].tolist())
print("First 10 predicted values:", y_pred[:10].tolist())
```

Task 9 — Evaluate Decision Tree

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

print("Decision Tree Model Evaluation")
print(f"Accuracy  : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"Recall    : {recall_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"F1-Score  : {f1_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print("\n", classification_report(y_test, y_pred, zero_division=0))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Decision Tree Confusion Matrix")
plt.tight_layout()
plt.show()
```

Task 10 — Compare maximum tree depth

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

results = {}
for depth in [3, 5, 7, 10, None]:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    score = accuracy_score(y_test, model.predict(X_test))
    results[str(depth)] = score
    print(f"Max depth {depth}: accuracy = {score:.4f}")

best_depth = max(results, key=results.get)
print("\nBest depth:", best_depth)
print("Best accuracy:", f"{results[best_depth]:.4f}")
```

Task 11 — 5-fold cross-validation

```
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
import numpy as np

model = DecisionTreeClassifier(max_depth=5, random_state=42)
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
print("Decision Tree Cross-Validation Results")
for i, score in enumerate(scores, 1):
    print(f"Fold {i} accuracy: {score:.4f}")
print("Mean accuracy:", f"{np.mean(scores):.4f}")
print("Standard deviation:", f"{np.std(scores):.4f}")
```

Topic 4 — Random Forest (HR Attrition example)

Example dataset: WA_Fn-UseC_-HR-Employee-Attrition.csv. Set target to Attrition. Drop identifiers and constant columns where present. Random Forest generally does not need scaling.

Dataset setup for this topic

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition.csv")
df.columns = df.columns.str.strip()
target = "Attrition"
df = df.drop(columns=[
    "EmployeeNumber", "EmployeeCount", "Over18", "StandardHours"
], errors="ignore")
df = df.dropna(subset=[target]).copy()

X = df.drop(columns=[target])
y = df[target].astype(str).str.strip()
X = pd.get_dummies(X, drop_first=True, dtype=int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("HR Attrition dataset ready:", X.shape)
print("Target distribution:")
print(y.value_counts())
```

Task 8 — Train Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100, random_state=42, class_weight="balanced"
)
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)

print("Random Forest trained successfully!")
print("Number of trees:", rf_model.n_estimators)
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("\nFirst 10 actual values:", y_test[:10].tolist())
print("First 10 predicted values:", y_pred[:10].tolist())
```

Task 9 — Evaluate Random Forest

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

print("Random Forest Model Evaluation")
print(f"Accuracy  : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"Recall    : {recall_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"F1-Score  : {f1_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print("\n", classification_report(y_test, y_pred, zero_division=0))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Random Forest Confusion Matrix")
plt.tight_layout()
plt.show()
```

Task 10 — Compare number of trees

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

results = {}
for n in [50, 100, 200]:
    model = RandomForestClassifier(
        n_estimators=n, random_state=42, class_weight="balanced"
    )
    model.fit(X_train, y_train)
    score = accuracy_score(y_test, model.predict(X_test))
    results[n] = score
    print(f"Trees = {n}: accuracy = {score:.4f}")

best_n = max(results, key=results.get)
print("\nBest number of trees:", best_n)
print("Best accuracy:", f"{results[best_n]:.4f}")
```

Task 11 — Cross-validation and feature importance

```
from sklearn.model_selection import cross_val_score
```

```
from sklearn.ensemble import RandomForestClassifier
import numpy as np
import pandas as pd

model = RandomForestClassifier(
    n_estimators=100, random_state=42, class_weight="balanced"
)
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
print("Random Forest 5-fold CV scores:", np.round(scores, 4))
print("Mean CV accuracy:", f"{np.mean(scores):.4f}")

model.fit(X_train, y_train)
importance = pd.Series(model.feature_importances_, index=X_train.columns)
print("\nTop 10 important features:")
print(importance.sort_values(ascending=False).head(10))
```

HR attrition datasets can be class-imbalanced. Consider recall, F1-score, and the confusion matrix, not accuracy alone.

Topic 5 — K-Nearest Neighbors (KNN)

KNN generally requires scaled features. Use the scaled classification template for Tasks 1–7.

Task 8 — Train KNN

```
from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
y_pred = knn_model.predict(X_test)

print("KNN model trained successfully!")
print("Number of neighbors (K):", knn_model.n_neighbors)
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("\nFirst 10 actual values:", y_test[:10].tolist())
print("First 10 predicted values:", y_pred[:10].tolist())
```

Task 9 — Evaluate KNN

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

print("KNN Model Evaluation")
print(f"Accuracy  : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"Recall    : {recall_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"F1-Score  : {f1_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print("\n", classification_report(y_test, y_pred, zero_division=0))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("KNN Confusion Matrix")
plt.tight_layout()
plt.show()
```

Task 10 — Compare K values

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

results = {}
for k in [1, 3, 5, 7, 9]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    score = accuracy_score(y_test, model.predict(X_test))
    results[k] = score
    print(f"K = {k}: accuracy = {score:.4f}")

best_k = max(results, key=results.get)
print("\nBest K:", best_k)
print("Best accuracy:", f"{results[best_k]:.4f}")
```

Task 11 — 5-fold cross-validation

```
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
print("KNN Cross-Validation Results")
for i, score in enumerate(scores, 1):
    print(f"Fold {i} accuracy: {score:.4f}")
print("Mean accuracy:", f"{np.mean(scores):.4f}")
print("Standard deviation:", f"{np.std(scores):.4f}")
```

Topic 6 — K-Means Clustering

K-Means is unsupervised: it does not require y or stratified splitting. Use numerical features only, handle missing values, encode categories if needed, and scale the full feature matrix before clustering.

Preparation and Task 8 — Fit K-Means

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv("your_dataset.csv")  # Change filename
df.columns = df.columns.str.strip()
df = df.drop(columns=["ID", "EmployeeNumber"], errors="ignore")
X = pd.get_dummies(df, drop_first=True, dtype=int)
X = X.fillna(X.median(numeric_only=True)).fillna(0)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

print("K-Means clustering completed!")
print("Number of clusters:", kmeans.n_clusters)
print("Data shape:", X_scaled.shape)
print("\nCluster labels (first 10):", clusters[:10])
print("\nCluster counts:")
print(pd.Series(clusters).value_counts().sort_index())
```

Task 9 — Evaluate clusters with inertia and silhouette score

```
from sklearn.metrics import silhouette_score

print("K-Means Evaluation")
print("------------------")
print("Inertia:", round(kmeans.inertia_, 4))
if len(set(clusters)) > 1 and len(set(clusters)) < len(clusters):
    print("Silhouette score:", round(silhouette_score(X_scaled, clusters), 4))
else:
    print("Silhouette score is not defined for this clustering result.")
```

Task 10 — Elbow method (compare K values)

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertias = []
k_values = range(1, 8)

for k in k_values:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertias.append(model.inertia_)
    print(f"K = {k}, inertia = {model.inertia_:.4f}")

plt.plot(list(k_values), inertias, marker="o")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.title("K-Means Elbow Method")
plt.tight_layout()
plt.show()
```

Task 11 — Compare silhouette scores

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

print("K-Means Silhouette Comparison")
best_k, best_score = None, -1
for k in range(2, min(8, len(X_scaled))):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    print(f"K = {k}, silhouette score = {score:.4f}")
    if score > best_score:
        best_k, best_score = k, score

print("\nBest K:", best_k)
print("Best silhouette score:", f"{best_score:.4f}")
```

PCA or clustering visuals are easier to interpret when plotting two dimensions. If you have a known class label, you may color the plot for visualization, but do not use that label to train K-Means.

Topic 7 — Linear Regression (Loan Amount Prediction)

Linear Regression predicts a continuous numerical value. Change the filename and TARGET to match the provided loan dataset. The target must be numeric.

Preparation and Task 8 — Train model

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv("loan.csv")  # Change filename
df.columns = df.columns.str.strip()
target = "LoanAmount"         # Change target if required
df = df.dropna(subset=[target]).copy()
df = df.drop(columns=["ID", "Loan_ID"], errors="ignore")

X = df.drop(columns=[target])
y = pd.to_numeric(df[target], errors="coerce")
valid = y.notna()
X, y = X.loc[valid], y.loc[valid]
X = pd.get_dummies(X, drop_first=True, dtype=int)
X = X.fillna(X.median(numeric_only=True)).fillna(0)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Linear Regression model trained successfully!")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("\nFirst 5 actual loan amounts:", y_test.head().tolist())
print("First 5 predicted amounts:", y_pred[:5].round(2).tolist())
```

Task 9 — Regression evaluation

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Linear Regression Evaluation")
print("----------------------------")
print(f"MAE  : {mae:.4f}")
print(f"MSE  : {mse:.4f}")
print(f"RMSE : {rmse:.4f}")
print(f"R2   : {r2:.4f}")
```

Task 10 — Compare Linear Regression with a baseline

```
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

models = {
    "Mean baseline": DummyRegressor(strategy="mean"),
    "Linear Regression": LinearRegression()
}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(f"{name}: MAE = {mean_absolute_error(y_test, pred):.4f}, "
          f"R2 = {r2_score(y_test, pred):.4f}")
```

Task 11 — 5-fold cross-validation (R^2)

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
import numpy as np

model = LinearRegression()
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="r2")
print("Linear Regression Cross-Validation (R2)")
for i, score in enumerate(scores, 1):
    print(f"Fold {i} R2: {score:.4f}")
print("Mean R2:", f"{np.mean(scores):.4f}")
print("Standard deviation:", f"{np.std(scores):.4f}")
```

For regression, do not use classification metrics or stratify=y. MAE, RMSE, and R^2 are common metrics. Negative R^2 can occur and means the model performed worse than a mean-prediction baseline on that evaluation set.

Topic 8 — Bagging Classifier

Bagging trains multiple base estimators on bootstrap samples and combines their predictions. This example uses Decision Trees as base estimators.

Task 8 — Train Bagging

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_model = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=42),
    n_estimators=10, random_state=42
)
bag_model.fit(X_train, y_train)
y_pred = bag_model.predict(X_test)

print("Bagging model trained successfully!")
print("Base estimator: Decision Tree")
print("Number of models:", bag_model.n_estimators)
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("\nFirst 10 actual values:", y_test[:10].tolist())
print("First 10 predicted values:", y_pred[:10].tolist())
```

Task 9 — Evaluate Bagging

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, ConfusionMatrixDisplay
)
import matplotlib.pyplot as plt

print("Bagging Model Evaluation")
print(f"Accuracy  : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"Recall    : {recall_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print(f"F1-Score  : {f1_score(y_test, y_pred, average='weighted', zero_division=0):.4f}")
print("\n", classification_report(y_test, y_pred, zero_division=0))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Bagging Confusion Matrix")
plt.tight_layout()
plt.show()
```

Task 10 — Compare estimator counts

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

results = {}
for n in [5, 10, 20, 50]:
    model = BaggingClassifier(
        estimator=DecisionTreeClassifier(random_state=42),
        n_estimators=n, random_state=42
    )
    model.fit(X_train, y_train)
    score = accuracy_score(y_test, model.predict(X_test))
    results[n] = score
    print(f"Models = {n}: accuracy = {score:.4f}")

best_n = max(results, key=results.get)
print("\nBest number of models:", best_n)
print("Best accuracy:", f"{results[best_n]:.4f}")
```

Task 11 — 5-fold cross-validation

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
import numpy as np

model = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=42),
    n_estimators=10, random_state=42
)
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
print("Bagging Cross-Validation Results")
for i, score in enumerate(scores, 1):
    print(f"Fold {i} accuracy: {score:.4f}")
print("Mean accuracy:", f"{np.mean(scores):.4f}")
print("Standard deviation:", f"{np.std(scores):.4f}")
```

This uses the current scikit-learn parameter name estimator=. Older scikit-learn versions may require base_estimator= instead.

Topic 9 — PCA (Principal Component Analysis)

PCA reduces feature dimensions while preserving as much variance as possible. It is not a classifier and does not require a target column. For text data such as English documents, vectorize the text before PCA; for this example, X must be numerical.

Preparation and Task 8 — Standardize and apply PCA

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

df = pd.read_csv("pca.csv")  # Change filename
df.columns = df.columns.str.strip()
df = df.drop(columns=["ID"], errors="ignore")
X = pd.get_dummies(df, drop_first=True, dtype=int)
X = X.fillna(X.median(numeric_only=True)).fillna(0)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

n_comp = min(2, X_scaled.shape[0], X_scaled.shape[1])
pca = PCA(n_components=n_comp)
X_pca = pca.fit_transform(X_scaled)

print("PCA applied successfully!")
print("Original shape:", X.shape)
print("Reduced shape:", X_pca.shape)
print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Total variance retained:",
      f"{pca.explained_variance_ratio_.sum() * 100:.2f}%")
print("\nFirst 5 reduced samples:")
print(X_pca[:5])
```

Task 9 — Visualize the first two components

```
import matplotlib.pyplot as plt

if X_pca.shape[1] >= 2:
    plt.figure(figsize=(7, 5))
    plt.scatter(X_pca[:, 0], X_pca[:, 1], alpha=0.7)
    plt.xlabel("Principal Component 1")
    plt.ylabel("Principal Component 2")
    plt.title("PCA Two-Dimensional Visualization")
    plt.tight_layout()
    plt.show()
    print("PCA visualization completed.")
else:
    print("Only one component is available; a 2D plot cannot be created.")
```

Task 10 — Explained variance

```
import numpy as np
import matplotlib.pyplot as plt

variance = pca.explained_variance_ratio_
print("PCA Explained Variance Analysis")
for i, value in enumerate(variance, 1):
    print(f"Principal Component {i}: {value:.4f}")
print("Total variance retained:", f"{np.sum(variance) * 100:.2f}%")

plt.bar(range(1, len(variance) + 1), variance)
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance Ratio")
plt.title("PCA Explained Variance")
plt.tight_layout()
plt.show()
```

Task 11 — Cumulative explained variance

```
from sklearn.decomposition import PCA
import numpy as np
import matplotlib.pyplot as plt

pca_full = PCA()
pca_full.fit(X_scaled)
cum = np.cumsum(pca_full.explained_variance_ratio_)

print("PCA Cumulative Explained Variance")
for i, value in enumerate(cum, 1):
    print(f"Components {i}: {value * 100:.2f}%")

n95 = np.searchsorted(cum, 0.95) + 1
print("\nComponents needed for at least 95% variance:", n95)

plt.plot(range(1, len(cum) + 1), cum, marker="o")
```

```
plt.axhline(0.95, linestyle="--", label="95% variance")
plt.xlabel("Number of components")
plt.ylabel("Cumulative explained variance")
plt.title("PCA Cumulative Explained Variance")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

For PCA, fit scaling and PCA on training data only when your goal is downstream prediction. The standalone unsupervised illustration above fits the supplied dataset to demonstrate dimensionality reduction.

Quick revision: key differences

Algorithm	Learning type	Scaling?	Typical evaluation
SVM	Supervised classification	Yes	Accuracy, precision, recall, F1
MLP	Supervised classification	Yes	Accuracy, precision, recall, F1
Decision Tree	Supervised classification	Usually no	Classification metrics
Random Forest	Supervised classification	Usually no	Classification metrics
KNN	Supervised classification	Yes	Classification metrics
K-Means	Unsupervised clustering	Usually yes	Inertia, silhouette score
Linear Regression	Supervised regression	Optional	MAE, RMSE, R^2
Bagging	Ensemble (here: classification)	Usually no for trees	Classification metrics
PCA	Dimensionality reduction	Yes	Explained variance

Common troubleshooting

1. NameError: run the earlier cell that creates the variable (for example, X_train or target).

2. KeyError for a target: print(df.columns.tolist()) and set target to the exact column name in your CSV.

3. ValueError converting text to numbers: encode categorical features or vectorize text before training.

4. Stratify error: stratified splitting needs enough samples per class; for very small datasets, remove stratify=y.

5. PCA/ K-Means do not require y. Linear Regression requires a numerical y. Do not use classification evaluation metrics for regression.

6. In a notebook, run cells in order after a kernel restart. Do not repeatedly scale the already scaled data.

Exam workflow: Load → inspect → clean → prepare features/target → encode → split → scale when appropriate → train → evaluate → compare parameters → cross-validate / analyze.